Задача 1. Задача Иосифа

Источник:	базовая
Имя входного файла:	input.txt
Имя выходного файла:	output.txt
Ограничение по времени:	1 секунда
Ограничение по памяти:	разумное

В первой строке входного файла записаны два целых числа N и K ($1 \le N \le 10^5$, $1 \le K \le 10^9$), где N — количество людей, стоящих в кругу, а K — шаг исключения. Люди пронумерованы от 1 до N и стоят в кругу. Начиная с человека номер 1, каждый K-й человек исключается из круга, после чего отсчет продолжается с следующего человека. Требуется вывести в выходной файл номер последнего оставшегося человека.

Пример

input.txt	output.txt
5 2	3

Пояснение к примеру

В кругу стоит 5 человек, исключение идет с шагом 2.

- Начинаем с человека 1: исключаем человека 2 (1 → 2).
- Следующий отсчет с человека 3: исключаем человека 4 (3 → 4).
- Следующий отсчет с человека 5: исключаем человека 1 (5 → 1).
- Следующий отсчет с человека 3: исключаем человека 5 (3 → 5).
- Остается человек 3.

Ответ: 3.

Задача 2. Растущий массив

Источник:	базовая
Имя входного файла:	input.txt
Имя выходного файла:	output.txt
Ограничение по времени:	1 секунда
Ограничение по памяти:	разумное

В данной задаче нужно реализовать массив переменного размера, в который можно дописывать элементы, не зная заранее его окончательный размер. Используя эту структуру данных, нужно решить приведённую ниже задачу.

В первой строке записано целое число N — количество записей ($1 \le N \le 2 \cdot 10^5$). В остальных N строках содержатся записи, по одной в строке.

Для каждой записи указаны ключ и значение через пробел. Ключ — это целое число в диапазоне от 0 до 10^6 включительно, а значение — это строка от одного до семи символов включительно, состоящая только из маленьких букв латинского алфавита.

Требуется вывести ровно те же самые N записей, но в другом порядке. Записи должны быть упорядочены по возрастанию ключа. Если у нескольких записей ключ равный, то нужно упорядочить их в том порядке, в котором они встречаются по входном файле.

Важно: решать задачу **нужно** следующим образом (другие решения засчитываться не будут). Нужно завести 10^6 **массивов** переменного размера, и в каждый k-ый массив складывать все записи с ключом, равным k. После раскусывания записей по массивам достаточно будет пробежаться по массивам в порядке увеличения k и распечатать их..

Пример

input.txt	output.txt
7 3 qwerty 3 string 6 good 1 a 3 ab 2 hello 5 world	1 a 2 hello 3 qwerty 3 string 3 ab 5 world 6 good

Пояснение к примеру

В примере 7 записей с ключами 1, 2, 3, 5 и 6 — именно в таком порядке записи и выведены в выходном файле. Обратите внимание, что есть три записи с ключом 3: qwerty, string, ab. Они выведены ровно в том порядке, в котором они идут во входном файле.

Задача 3. Тасовка перфокарт

Источник:	базовая
Имя входного файла:	input.txt
Имя выходного файла:	output.txt
Ограничение по времени:	1 секунда
Ограничение по памяти:	**специальное**

Имеется две колоды перфокарт: левая колода и правая. Изначально в каждой колоде ровно N перфокарт. В левой колоде перфокарты пронумерованы числами от 1 до N по порядку, если просматривать их сверху вниз. В правой колоде перфокарты пронумерованы числами от -1 до $-N$ по порядку, если просматривать их сверху вниз.

Для перемешивания колод нужно выполнить M заданных операций, каждая операция заключается в перекладывании одной карты. Каждая операция обозначается одной шестнадцатеричной цифрой в диапазоне от 0 до F (15) включительно. Операция определяется значениями четырёх битов в двоичной записи этой цифры:

- Если старший бит (8) единичный, то нужно взять карту с правой колоды, а иначе — с левой колоды.
- Если предпоследний бит (4) единичный, то нужно взять карту снизу колоды, а иначе — сверху колоды.
- Если второй бит (2) единичный, то нужно положить карту в правую колоды, а иначе — в левую.
- Если младший бит (1) единичный, то нужно положить карту в колоду снизу, а иначе — сверху.

Если в какой-то момент нужно выполнить операцию, которая предписывает взять карту из пустой колоды, то такую операцию нужно пропустить (ничего не делая).

В первой строке записано два целых числа: N — начальное количество карт в каждой колоде и M — сколько операций нужно выполнить ($1 \le N \le 5 \cdot 10^5$, $0 \le M \le 5 \cdot 10^6$).

Во второй строке записано подряд ровно M символов — описание операций в порядке их выполнения. Каждый символ является шестнадцатеричной цифрой и изменяется в диапазоне от 0 до 9 или от A до F включительно.

После выполнения всех операций требуется вывести содержимое левой колоды в первой строке выходного файла, и содержимое правой колоды — во второй строке. В каждой строке нужно сначала вывести целое число K — количество карт в колоде после выполнения всех операций, а затем через пробел K целых чисел — номера перфокарт в колоде, перечисленные в порядке сверху вниз.

Важно: требуется хранить каждую колоду в **кольцевом буфере** размером ровно на $(2N + 1)$ элементов. Память под кольцевые буферы выделяйте динамически.

Примеры

input.txt	output.txt
5 0	5 1 2 3 4 5 5 -1 -2 -3 -4 -5
5 10 180FA45DB2	6 -1 2 3 4 5 -5 4 1 -3 -4 -2
3 20 CCCCCCCCCC9999999999	6 -1 -2 -3 1 2 3 0

Комментарий

По второму примеру можно заметить, что команды 0, F, A и 5 никогда ничего не изменяют.

Задача 4. Число атомов

Источник:	основная
Имя входного файла:	input.txt
Имя выходного файла:	output.txt
Ограничение по времени:	1 секунда
Ограничение по памяти:	разумное

Задана строка, formula, представляющая химическую формулу, вернуть *количество каждого атома*. Атомарный элемент всегда начинается с заглавной буквы, затем следует ноль или более строчных букв, представляющих имя. Если количество элементов больше, за ним может следовать одна или несколько цифр, представляющих их количество 1. Если количество равно 1, за ним не следует ни одной цифры.

- Например, "H_2O"и "H_2O_2"возможны, но "H_1O_2"невозможно.

Две формулы объединяются вместе для получения еще одной формулы.

1. Например, "$H_2O_2He_3Mg_4$"— это тоже формула.

Формула, заключенная в скобки, и количество (добавленное по желанию) также являются формулой.

- Например, "(H_2O_2)"и "$(H_2O_2)_3$"— это формулы.

Верните количество всех элементов в виде строки в следующей форме: первое имя (в отсортированном порядке), затем его количество (если это количество больше 1), затем второе имя (в отсортированном порядке), затем его количество (если это количество больше 1) и т. д.

Тестовые случаи генерируются таким образом, чтобы все значения на выходе умещались в **32-битное** целое число.

Формат входного файла

Длина формулы не превышает 1000 ($1 \le$ formula_lenght $\le 300\,000$).

Примеры

input.txt	output.txt
H2O	H2O
Mg(OH)2	H2MgO2
K4(ON(SO3)2)2	K4N2O14S4

Задача 5. Волшебники

Источник:	основная
Имя входного файла:	input.txt
Имя выходного файла:	output.txt
Ограничение по времени:	1 секунда
Ограничение по памяти:	разумное

Будучи правителем королевства, вы имеете в своем распоряжении целую армию волшебников.
Вам дан массив целых чисел **с силой волшебника** strength. Для **непрерывной** группы волшебников (т.е. силы волшебников образуют подмассив **)** общая **сила** определяется как **произведение** следующих двух значений:

- Сила самого **слабого** волшебника в группе.
- Сумма индивидуальных сил **всех** волшебников в группе.

Верните *сумму сил **всех** смежных групп волшебников*. Поскольку ответ может быть очень большим, верните его ***по*** **модулю** $10^9 + 7$
Подмассив — это непрерывная **непустая** последовательность элементов внутри массива.

Примеры

input.txt	output.txt
1 3 1 2	44
5 4 6	213

Пояснение к примеру 1

Ниже приведены все смежные группы волшебников:

- [1] из [**1** ,3,1,2] имеет общую силу min([1]) * sum([1]) = 1 * 1 = 1
- [3] из [1, **3** ,1,2] имеет общую силу min([3]) * sum([3]) = 3 * 3 = 9
- [1] из [1,3, **1** ,2] имеет общую силу min([1]) * sum([1]) = 1 * 1 = 1
- [2] из [1,3,1, **2**] имеет общую силу min([2]) * sum([2]) = 2 * 2 = 4
- [1,3] из [**1,3** ,1,2] имеет общую силу min([1,3]) * сумма([1,3]) = 1 * 4 = 4
- [3,1] из [1, **3,1** ,2] имеет общую силу min([3,1]) * сумма([3,1]) = 1 * 4 = 4
- [1,2] из [1,3, **1,2**] имеет общую силу min([1,2]) * сумма([1,2]) = 1 * 3 = 3
- [1,3,1] из [**1,3,1** ,2] имеет общую силу min([1,3,1]) * сумма([1,3,1]) = 1 * 5 = 5
- [3,1,2] из [1, **3,1,2**] имеет общую силу min([3,1,2]) * сумма([3,1,2]) = 1 * 6 = 6
- [1,3,1,2] из [**1,3,1,2**] имеет общую силу min([1,3,1,2]) * sum([1,3,1,2]) = 1 * 7 = 7

Сумма всех общих сил составляет 1 + 9 + 1 + 4 + 4 + 4 + 3 + 5 + 6 + 7 = 44.

Задача 6. Неубывающие подмассивы

Источник:	основная
Имя входного файла:	input.txt
Имя выходного файла:	output.txt
Ограничение по времени:	1 секунда
Ограничение по памяти:	разумное

Вам дан массив nums целых n чисел и целое число k.
К каждому подмассиву nums можно применить **до** k = 100 операций. Каждая операция увеличивает любой элемент подмассива на 1.
Обратите внимание: каждый подмассив рассматривается независимо, то есть изменения, внесенные в один подмассив, не сохраняются в другом.
Верните количество подмассивов, которые можно сделать **неубывающими** после выполнения большинства k операций.
Массив называется **неубывающим**, если каждый его элемент больше или равен предыдущему элементу, если таковой существует.

Формат входного файла

- В первой строке входного файла задано число k – количество операций ($1 \le k \le 10^9$).
- Во второй строку задан сам массив длинной $\le$ length $\le 10^5$

Примеры

input.txt	output.txt
7 6 3 1 2 4 4	17
4 6 3 1 3 6	12

Пояснение к примеру

1. Из всех 21 возможных подмассивов nums, только подмассивы [6, 3, 1], [6, 3, 1, 2], [6, 3, 1, 2, 4] и [6, 3, 1, 2, 4, 4] не могут быть сделаны неубывающими после применения до k = 7 операций. Таким образом, количество неубывающих подмассивов равно 21 - 4 = 17.
2. Подмассив, [3, 1, 3, 6], а также все подмассивы nums с тремя или менее элементами, за исключением [6, 3, 1], можно сделать неубывающими после k выполнения операций. Имеется 5 подмассивов из одного элемента, 4 подмассива из двух элементов и 2 подмассива из трёх элементов, за исключением [6, 3, 1], поэтому существуют 1 + 5 + 4 + 2 = 12 подмассивы, которые можно сделать неубывающими.

Задача 7. Ближайшие точки к началу координат.

Источник:	основная
Имя входного файла:	input.txt
Имя выходного файла:	output.txt
Ограничение по времени:	1 секунда
Ограничение по памяти:	разумное

Дан массив points, где points[i] = [x_i, y_i] представляет точку на плоскости **XY**, и целое число k, вернуть k ближайших точки к началу координат (0, 0).

Расстояние между двумя точками на плоскости **XY** называется евклидовым расстоянием (т. е. $\sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$).

Вы можете возвращать ответ в **любом порядке**. Ответ **гарантированно** будет **уникальным** (за исключением порядка, в котором он указан).

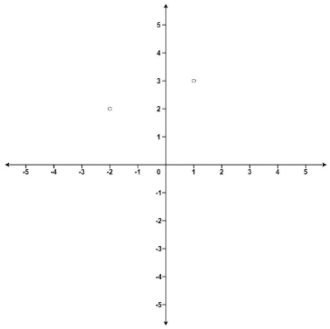
Формат входного файла

- В первой строке входного файла задано число k – количество точек, которые надо вывести ($1 \le k \le$ length_points $\le 10^4$).
- Далее задан сам массив попарно с новой строки

Примеры

input.txt	output.txt
1 1 3 -2 2	-2 2
2 3 3 5 -1 -2 4	-2 4 3 3

Пояснение к примеру 1



Расстояние между (1, 3) и началом координат равно sqrt(10). Расстояние между (-2, 2) и началом координат равно sqrt(8). Поскольку sqrt(8) < sqrt(10), (-2, 2) ближе к началу координат. Нам нужны только ближайшие к началу координат точки с k = 1, поэтому ответ — [[-2,2]].

Задача 8. Игра «Прыжки»

Источник:	повышенной сложности
Имя входного файла:	input.txt
Имя выходного файла:	output.txt
Ограничение по времени:	1 секунда
Ограничение по памяти:	разумное

Вам дан целочисленный массив nums и целое число k.

Изначально вы находитесь на индексе массива 0. За один ход вы можете сделать максимум k шагов вперёд, не выходя за границы массива. То есть вы можете перейти от индекса index = i к любому индексу в диапазоне index ∈ [i + 1, min(n - 1, i + k)].

Вам нужно достичь последнего индекса массива (index = n - 1). Ваш **результат** — это **сумма** всех посещённых вами nums[j]индексов j в массиве.

Верните *максимально возможный **балл***.

Формат входного файла

- В первой строке входного файла задано число k – количество шагов за ход ($1 \le k \le$ length_arr $\le 10^5$).
- Во второй строку задан сам массив длинной length_arr

Примеры

input.txt	output.txt
2 1 -1 -2 4 -7 3	7
3 10 -5 -2 4 0 3	17

Пояснение к примеру

1. Вы можете выбрать свои прыжки, формируя подпоследовательность [1, -1, 4, 3] ([1 , -1 , -2, 4 , -7, 3]). Сумма равна 7.
2. Вы можете выбрать свои прыжки, формируя подпоследовательность [10, 4, 3] ([10 ,-5,-2, 4 ,0, 3]). Сумма равна 17.

Задача 9. Скобки

Источник:	повышенной сложности
Имя входного файла:	input.txt
Имя выходного файла:	output.txt
Ограничение по времени:	2 секунды
Ограничение по памяти:	разумное

Как многим известно, основу синтаксиса LISP-подобных языков составляют скобочки.

Круглые, фигурные, квадратные — на любой вкус. Конечно, там применяют и другие символы, но главное — скобочки.

В лабораториях Ну Гляди-какого-умного Университета (НГУ) придумали новый язык программирования — Uncommon LISP. Он впитал в себя самую суть всех функциональных языков. В нём кроме скобочек нет ничего...

В первой версии языка корректной программой на Uncommon LISP считалась правильная скобочная последовательность из четырех видов скобок: (), {}, [] и <>. Правильная скобочная последовательность определяется следующим образом:

- пустая строка — правильная скобочная последовательность;
- правильная скобочная последовательность, взятая в скобки одного типа — правильная скобочная последовательность;
- правильная скобочная последовательность, к которой приписана слева или справа правильная скобочная последовательность — тоже правильная скобочная последовательность.

К сожалению, эти правила были слишком сложными, так что во второй версии языка (Uncommon LISP v2) корректными программами стали считаться также те программы, из которых можно получить правильную скобочную последовательность, переставляя скобки определённым образом. В последовательности можно переставлять местами скобки, стоящие рядом, если они обе открывающие или обе закрывающие. Такую операцию можно выполнять сколько угодно раз, в том числе по несколько раз переставляя одни и те же скобки.

Необходимо определить для данной скобочной последовательности, является ли она корректной программой для Uncommon LISP v2.

Формат входного файла

В первой строке входного файла записано целое число N — количество тестов, следующих ниже. Каждый тест — это непустая строка, состоящая из скобок. Суммарная длина всех строк не превышает 1 000 000.

Формат выходного файла

В выходной файл необходимо для каждого теста в отдельную строку вывести ответ T, если заданную скобочную последовательность можно привести к правильному виду, и NIL, если нельзя.

Пример

input.txt	output.txt
2 ([)])([]	T NIL